

SQL FUNCTIONS

Ing. Ladislav Körösi, PhD.
ÚRK, FEI STU

`ladislav.korosi@stuba.sk`

SQL Aggregate Functions

SQL aggregate functions return a single value, calculated from values in a column.

Useful aggregate functions:

- AVG() - Returns the average value
- COUNT() - Returns the number of rows
- MAX() - Returns the largest value
- MIN() - Returns the smallest value
- SUM() - Returns the sum

SQL Scalar functions

SQL scalar functions return a single value, based on the input value.

Useful scalar functions:

- UCASE() - Converts a field to upper case
- LCASE() - Converts a field to lower case
- MID() - Extract characters from a text field
- LENGTH() - Returns the length of a text field
- ROUND() - Rounds a numeric field to the number of decimals specified
- NOW() - Returns the current system date and time
- FORMAT() - Formats how a field is to be displayed

AVG

The AVG() function returns the average value of a numeric column.

Syntax:

```
SELECT AVG(column_name) FROM table_name
```

AVG

Example:

The following SQL statement gets the average value of the "Price" column from the "Products" table:

```
SELECT AVG(Price) AS PriceAverage FROM Products;
```

Example:

The following SQL statement selects the "ProductName" and "Price" records that have an above average price:

```
SELECT ProductName, Price FROM Products  
WHERE Price > (SELECT AVG(Price) FROM Products);
```

COUNT

The COUNT() function returns the number of rows that matches a specified criteria.

Syntax:

```
SELECT COUNT(column_name) FROM table_name
```

```
SELECT COUNT(*) FROM table_name;
```

```
SELECT COUNT(DISTINCT column_name)  
FROM table_name;
```

COUNT

Example:

The following SQL statement counts the number of orders from "CustomerID"=7 from the "Orders" table:

```
SELECT COUNT(CustomerID) AS OrdersFromCustomerID7  
FROM Orders WHERE CustomerID=7;
```

Example:

The following SQL statement counts the total number of orders in the "Orders" table:

```
SELECT COUNT(*) AS NumberOfOrders FROM Orders;
```

MAX

The MAX() function returns the largest value of the selected column.

Syntax:

```
SELECT MAX(column_name) FROM table_name
```


MAX

Example:

The following SQL statement gets the largest value of the "Price" column from the "Products" table:

```
SELECT MAX(Price) AS HighestPrice FROM Products;
```

MIN

The MIN() function returns the smallest value of the selected column.

Syntax:

```
SELECT MIN(column_name) FROM table_name
```

MIN

Example:

The following SQL statement gets the smallest value of the "Price" column from the "Products" table:

```
SELECT  
    MIN(Price) AS SmallestOrderPrice  
FROM  
    Products;
```

SUM

The SUM() function returns the total sum of a numeric column.

Syntax:

```
SELECT SUM(column_name) FROM table_name
```

SUM

Example:

The following SQL statement finds the sum of all the "Quantity" fields for the "OrderDetails" table:

```
SELECT
    SUM(Quantity) AS TotalItemsOrdered
FROM
    OrderDetails;
```

UCASE

The UCASE() function converts the value of a field to uppercase.

Syntax:

```
SELECT UCASE(column_name) FROM table_name
```

UCASE

Example:

The following SQL statement selects the "CustomerName" and "City" columns from the "Customers" table, and converts the "CustomerName" column to uppercase:

```
SELECT UCASE(CustomerName) AS Customer, City  
FROM Customers;
```

LCASE

The LCASE() function converts the value of a field to lowercase.

Syntax:

```
SELECT LCASE(column_name) FROM table_name
```


LCASE

Example:

The following SQL statement selects the "CustomerName" and "City" columns from the "Customers" table, and converts the "CustomerName" column to lowercase:

```
SELECT LCASE(CustomerName) AS Customer, City  
FROM Customers;
```

MID

The MID() function is used to extract characters from a text field.

Syntax:

```
SELECT
    MID(column_name,start,length) AS some_name
FROM
    table_name;
```

MID

Parameter	Description
column_name	<u>Required</u> . The field to extract characters from
start	<u>Required</u> . Specifies the starting position (starts at 1)
length	<u>Optional</u> . The number of characters to return. If omitted, the MID() function returns the rest of the text

MID

Example:

The following SQL statement selects the first four characters from the "City" column from the "Customers" table:

```
SELECT MID(City,1,4) AS ShortCity  
FROM Customers;
```

LENGTH

The LENGTH() function returns the length of the value in a text field.

Syntax:

```
SELECT LENGTH(column_name) FROM table_name;
```

LENGTH

Example:

The following SQL statement selects the "CustomerName" and the length of the values in the "Address" column from the "Customers" table:

```
SELECT
    CustomerName,
    LENGTH(Address) as LengthOfAddress
FROM
    Customers;
```

ROUND

The ROUND() function is used to round a numeric field to the number of decimals specified.

Syntax:

```
SELECT ROUND(column_name,decimals)
FROM table_name;
```

Parameter	Description
column_name	<u>Required</u> . The field to round.
decimals	<u>Required</u> . Specifies the number of decimals to be returned.

ROUND

Note: Many database systems do rounding differently than you might expect. When rounding a number with a fractional part to an integer, our school teachers told us to round .1 through .4 DOWN to the next lower integer, and .5 through .9 UP to the next higher integer. But if all the digits 1 through 9 are equally likely, this introduces a slight bias towards infinity, since we always round .5 up. Many database systems have adopted the IEEE 754 standard for arithmetic operations, according to which the default rounding behavior is "round half to even." In this scheme, .5 is rounded to the nearest even integer. So, both 11.5 and 12.5 would be rounded to 12. The ROUND() function is used to round a numeric field to the number of decimals specified.

ROUND

Syntax:

The following SQL statement selects the product name and rounds the price in the "Products" table:

```
SELECT
```

```
    ProductName, ROUND(Price,0) AS RoundedPrice
```

```
FROM
```

```
    Products;
```

NOW

The NOW() function returns the current system date and time.

Syntax:

```
SELECT NOW() FROM table_name;
```

NOW

Example:

The following SQL statement selects the product name, and price for today from the "Products" table:

```
SELECT ProductName, Price, Now() AS PerDate  
FROM Products;
```

FORMAT

The FORMAT() function is used to format how a field is to be displayed.

Syntax:

```
SELECT FORMAT(column_name,format)
FROM table_name;
```

Parameter	Description
column_name	<u>Required</u> . The field to be formatted.
format	<u>Required</u> . Specifies the format.

FORMAT

Example:

The following SQL statement selects the product name, and price for today (formatted like YYYY-MM-DD) from the "Products" table:

```
SELECT
    ProductName,
    Price, FORMAT(Now(),'YYYY-MM-DD') AS PerDate
FROM
    Products;
```

HAVING

The HAVING clause was added to SQL because the WHERE keyword could not be used with aggregate functions.

Syntax:

SELECT

column_name, aggregate_function(column_name)

FROM table_name

WHERE column_name operator value

GROUP BY column_name

HAVING

aggregate_function(column_name) operator value;

HAVING

Example:

Ordes

OrderID	CustomerID	EmployeeID	OrderDate	ShipperID
10248	90	5	1996-07-04	3
10249	81	6	1996-07-05	1
10250	34	4	1996-07-08	2

Employees

EmployeeID	LastName	FirstName	BirthDate	Photo
1	Davolio	Nancy	1968-12-08	EmpID1.pic
2	Fuller	Andrew	1952-02-19	EmpID2.pic
3	Leverling	Janet	1963-08-30	EmpID3.pic

HAVING

Example:

Now we want to find if any of the employees has registered more than 10 orders.

```
SELECT Employees.LastName, COUNT(Orders.OrderID)  
AS NumberOfOrders
```

```
FROM (Orders INNER JOIN Employees  
ON Orders.EmployeeID=Employees.EmployeeID)
```

```
GROUP BY LastName
```

```
HAVING COUNT(Orders.OrderID) > 10;
```


HAVING

Example:

Now we want to find if the employees "Davolio" or "Fuller" have registered more than 25 orders.

```
SELECT Employees.LastName, COUNT(Orders.OrderID)
AS NumberOfOrders FROM Orders
INNER JOIN Employees
ON Orders.EmployeeID=Employees.EmployeeID
WHERE LastName='Davolio' OR LastName='Fuller'
GROUP BY LastName
HAVING COUNT(Orders.OrderID) > 25;
```